

Customer Personality Analysis and Machine Learning Report

Step 1: Dataset Search

1.1 Dataset Name:

Customer Personality Analysis Dataset

1.2 Source:

The dataset was obtained from Kaggle: <https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis>

1.3 Dataset Description:

The *Customer Personality Analysis Dataset* provides detailed information about customers, including demographic characteristics, purchasing behavior, and responses to marketing campaigns. This dataset is commonly used to analyze customer segmentation and marketing effectiveness.

The dataset consists of:

- Number of rows: 2240 observations
- Number of columns: 29 variables

The dataset includes a variety of data types:

- Numerical variables:
Examples include *Year_Birth*, *Income*, and product spending variables such as *MntWines*, *MntMeatProducts*, and *MntFruits*, which represent the amount spent by customers on different product categories.
- Categorical variables:
Examples include *Education* and *Marital_Status*, which describe customer demographic characteristics.

- Date variable:

Dt_Customer, representing the date when a customer enrolled with the company.

The diversity of variables enables comprehensive analysis of customer profiles and purchasing patterns.

1.4 Justification for Choosing the Dataset:

The choice of such data set is explained by the following factors:

To begin with, the dataset is quite relevant in real-life business applications. It is concerned with the customer's demographics and buying behavior, which is essential to the business decision-making process that includes customer segmentation, personalized marketing, and optimization of the campaign.

Secondly, the dataset is quite machine learning friendly. It has several independent variables (e.g., income, age, education level) and the outcome-related variables (e.g., product spending and campaign responses), and it is possible to develop the predictive models.

Thirdly, the data has the right level of complexity and diversity as it has both numeric and nominal variables. It allows the use of multiple data preprocessing methods, exploratory data analysis (EDA), and modeling methods, expressing the actual data situations in the real world.

Lastly, the dataset meets the requirements of the project regarding the size of the dataset, 2240, which is within the necessary range of 1000-5000 rows.

In general, this data has a realistic and useful environment of conducting data analysis and machine learning activities. What's more, this dataset enables businesses to identify high-value customers and optimize marketing strategies, directly contributing to improved revenue and customer retention.

1.5 Machine Learning Task:

Based on the characteristics of the dataset, the following machine learning tasks can be identified:

- Regression Task:

Predicting customer spending behavior (e.g., total amount spent) based on demographic and socioeconomic factors such as income and age.

- **Classification Task:**
Predicting whether a customer will respond to a marketing campaign based on their profile and past behavior.

These tasks are relevant for supporting data-driven business decisions and improving marketing strategies.

1.6 Suitable Algorithm:

To address the identified tasks, the following machine learning algorithms are considered appropriate:

- **Linear Regression:**
Suitable for modeling and predicting continuous variables such as customer spending. It helps in understanding the relationship between independent variables and the target variable.
- **Logistic Regression and other Classification Models:**
Suitable for classification problems, such as predicting customer responses to marketing campaigns. These models can effectively estimate the probability of a binary outcome.

The selected algorithms are appropriate due to their interpretability, efficiency, and suitability for structured datasets with both numerical and categorical variables.

Step 2: Explanation of Each Library

`os`: Used to retrieve the current working directory and manage file paths, ensuring datasets can be read correctly.

`pandas (pd)`: A core data processing library used for loading data, cleaning data, conducting exploratory data analysis, and performing tabular data operations.

`platform`: Retrieves operating system and environment information for code compatibility checks and documentation.

`matplotlib.pyplot (plt)`: Plots basic charts to visualize data distributions, trends, and comparative relationships.

seaborn (sns): Creates more aesthetically pleasing statistical charts to explore variable relationships and identify outliers.

scipy.stats: Used for hypothesis testing, calculating t-statistics and p-values to support business decision-making analysis.

train_test_split: Splits datasets into training and testing sets for machine learning model training and evaluation.

LogisticRegression: Implements the logistic regression classification algorithm for binary classification prediction in business scenarios.

accuracy_score: Calculates the prediction accuracy of classification models to evaluate model performance.

Step 3: Read Dataset

3.1 Introduction and get the Dataset:

The selected dataset, *marketing_campaign.csv*, was obtained from Kaggle and contains 2240 rows and 29 columns, including both numerical and categorical variables. This satisfies the assignment requirement of having between 1000 and 5000 observations.

The dataset was loaded into Python using the Pandas library, ensuring proper handling of encoding and memory usage. This step ensures that the data is correctly imported and ready for further processing.

Related code:

```

import os
print(os.getcwd())
import platform
df=pd.read_csv(r"marketing_campaign.csv")

def load_and_preprocess_data(file_path):
    try:
        # Read CSV file
        df = pd.read_csv(
            file_path,
            encoding='utf-8',
            low_memory=False
        )
        print("\n===Dataset loaded successfully!===")
    # Catch FileNotFoundError
    except FileNotFoundError:

        desktop_path = r"~"
        dataset_name = os.path.basename(file_path)

        print(f"\n Error: '{dataset_name}' not found on desktop!")
        for file in os.listdir(desktop_path):
            print(f"    - {file}")
        exit() # Terminate the program to avoid errors in subsequent code

```

Figure1: Code for Geting Dataset

3.2 Basic validation is performed after import:

1. Row/column count: 2240 rows and 29 columns (row count meets requirements);
2. Data preview: df.head() confirms normal data structure;
3. Type validation: df.dtypes and select_dtypes() distinguish numerical and categorical columns to clarify data features.

Related code:

```

print("\n===Basic Dataset Validation===")

#Check dataset row count (assignment requirement: 1000-5000 rows)
row_count = len(df)
col_count = len(df.columns)
print(f"Total rows: {row_count}")
print(f"Total columns: {col_count}")
# Verify if row count meets the requirements
if 1000 <= row_count <= 5000:
    print("Row count meets assignment requirements (1000-5000 rows)")
else:
    print(f"Warning: Row count ({row_count}) does not meet requirements (1000-5000 rows)")

# View first 5 rows of data
print("\n=== First 5 Rows ===")
print(df.head())

# View data types
print("\n=== Data Types ===")
print(df.dtypes)

# Count number of numeric/categorical columns
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()
print(f"\nNumeric columns ({len(numeric_cols)}): {numeric_cols}")
print(f"Categorical columns ({len(categorical_cols)}): {categorical_cols}")

```

Figure2: Code for Basic Dataset Validation

3.3 Necessary preprocessing ensures data quality:

1. Missing values: Income column missing values filled with mean to retain data distribution;
2. Duplicates: `df.duplicated()` detects and removes duplicate rows to avoid analysis bias;
3. Date optimization: `Dt_Customer` column converted from string to datetime for time series analysis.

Related code:

```

print("\n===Targeted Preprocessing===")

# Missing value handling
print("\n1. Missing Value Handling")
missing_count = df.isnull().sum()
missing_cols = missing_count[missing_count > 0].index.tolist()

if missing_cols:
    print(f"Columns with missing values: {missing_cols}")
    # Fill numeric columns with mean
    for col in missing_cols:
        if col in numeric_cols:
            fill_value = df[col].mean()
            df[col].fillna(fill_value, inplace=True)
            print(f"Filled '{col}' with mean: {fill_value:.2f}")
else:
    print("No missing values in dataset")

# Duplicate value handling
print("\n2. Duplicate Handling")
duplicate_count = df.duplicated().sum()
if duplicate_count > 0:
    df.drop_duplicates(inplace=True)
    print(f"Removed {duplicate_count} duplicate rows")
else:
    print("No duplicate rows")

# Data format optimization
print("\n3. Date Format Optimization")
if "Dt_Customer" in df.columns:
    df["Dt_Customer"] = pd.to_datetime(df["Dt_Customer"], format='%d-%m-%Y')
    print("Converted 'Dt_Customer' to datetime format")
    print("Sample of 'Dt_Customer':")
    print(df["Dt_Customer"].head())

```

Figure3: Code for Targeted Preprocessing

3.4 Validation after preprocessing:

1. Missing values: Total missing values = 24;
2. Verify: final datasize=2240 rows $\times$ 29 columns

Related code:

```

print("\n===Preprocessing Result Validation===")

final_missing = df.isnull().sum().sum()
print(f"Total missing values after preprocessing: {final_missing}")

# Verify final dataset size
final_row_count = len(df)
print(f"Final dataset size: {final_row_count} rows × {df.columns.size} columns")

```

Figure4: Code for Preprocessing Result Validation

Step 4: Exploratory Data Analysis (EDA) and Data Processing

4.1 Introduction

In this section, conduct exploratory data analysis (EDA) to understand customer behavior and consumption patterns. Analysis includes summary statistics, data visualization, data cleaning and the use of programming logic, such as conditional statements, loops and functions. The goal is to extract meaningful business insights to support decision-making.

4.2 Data cleaning and processing

Before the analysis, data is preprocessed to ensure data quality and consistency.

First, missing values were handled in the preprocessing stage by replacing them with appropriate values (e.g., mean for Income), ensuring data completeness and consistency.

Secondly, a new variable called Total_Spending was created by adding customer expenses in multiple product categories such as wine, fruit, meat, fish, candy and gold products. This variable represents the overall consumption behavior of each customer, which is crucial for further analysis.

4.3 Summary statistics

Summary statistics are generated for key variables such as income and total expenditure. The results show that there is a big difference in customer income, indicating that there are different income groups. Similarly, the total expenditure also shows a high degree of variability, which shows that customers' purchasing behaviors are diverse.

This change provides opportunities for customer segmentation and targeted marketing strategies.

4.4 Data visualization and insights

4.4.1 Distribution of total expenditure

The histogram is used to visualize the distribution of total expenditure.

The results show that the consumption level of most customers is relatively low to medium, while only a small number of customers show very high spending. This shows that the company relies on a limited number of high-value customers to obtain a large part of its revenue.

Business insight:

The company should focus on retaining high-consumption customers and formulate strategies to increase the expenditure of low- and medium-low customers.

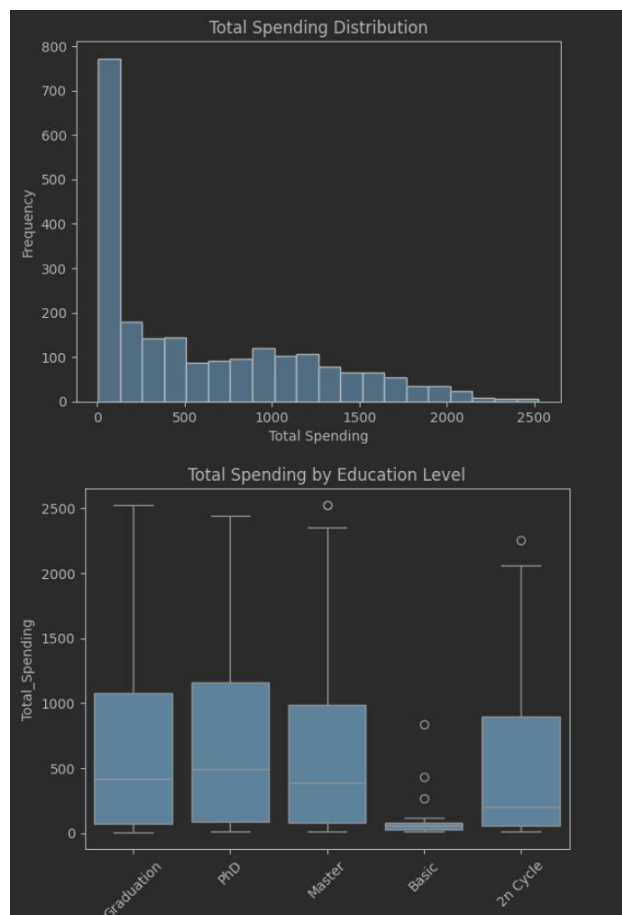


Figure5: Total spending distribution and Total spending by education level

4.4.2 Total expenditure by education level

Box charts are used to compare expenditures at different levels of education.

Analysis shows that customers with higher education tend to have higher median spending than others. This may be related to higher income levels or different consumption preferences.

Business insight:

Marketing activities can be customized according to the level of education, and high-end products are aimed at highly educated customers.

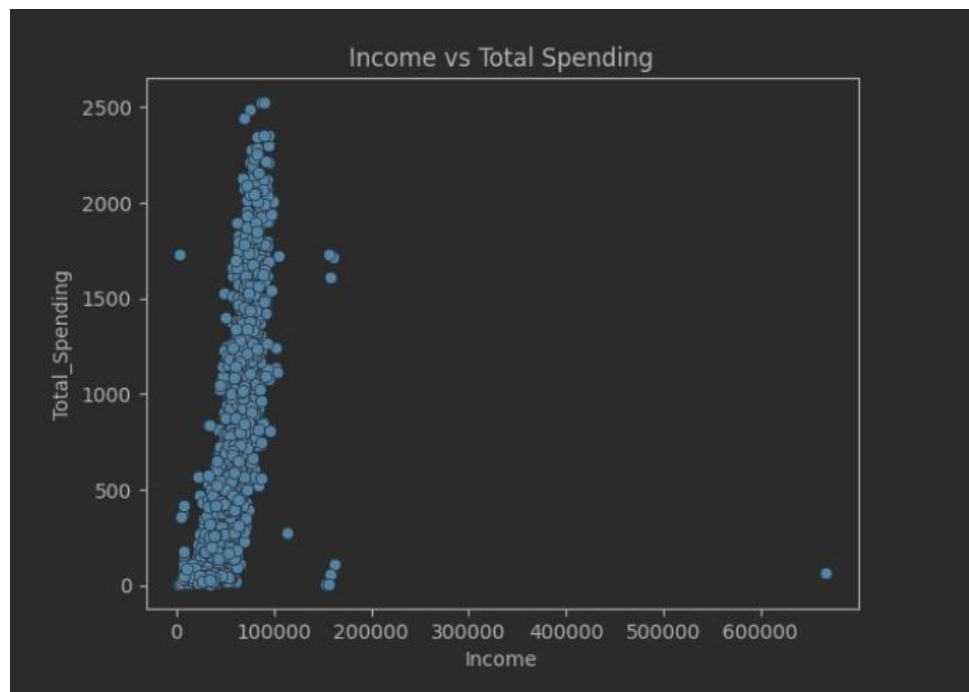


Figure6: Income vs Total spending

4.4.3 The relationship between income and expenditure

Scatter diagrams are used to check the relationship between income and total expenditure.

The results reveal that there is a positive relationship between income and expenditure, indicating that customers with higher income tend to spend more.

Business insight:

Revenue is the key driving force of customer spending. Companies should give priority to high-income customers in promotional activities and loyalty programs.

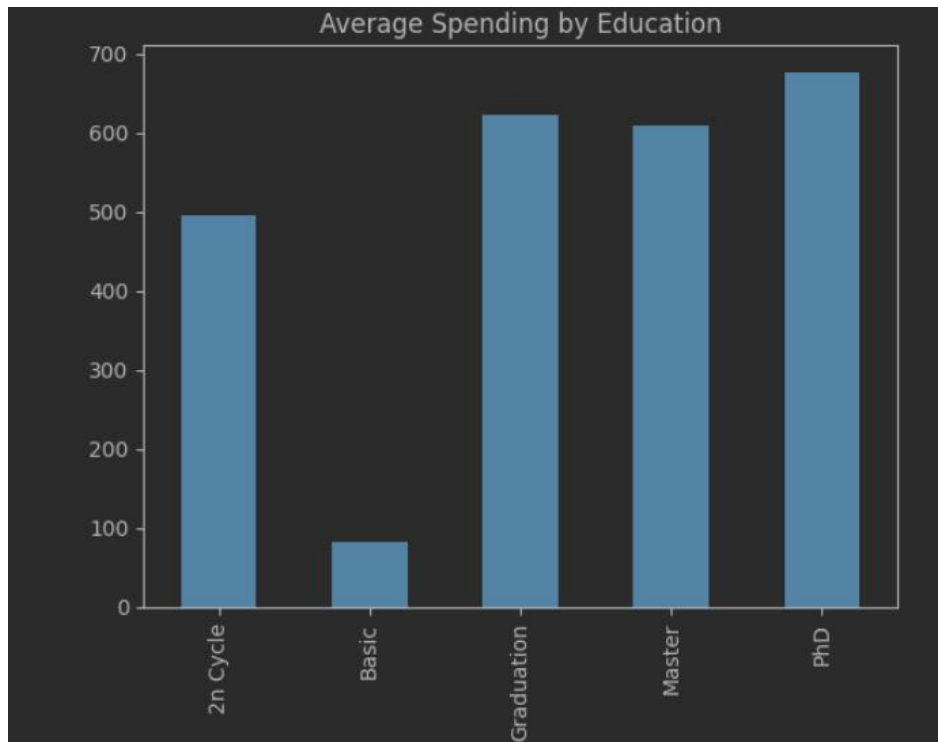


Figure7: Average Spending by Education

4.5 The use of programming logic

4.5.1 Conditional statements

Conditional statement (if-elif-else) is used to divide customers into different expenditure levels (high, medium and low) according to their total expenditure.

This classification helps to subdivide customers into meaningful groups for targeted marketing.

4.5.2 Cycle

The cycle is used to calculate the total expenditure of all customers. This shows how to use the programming structure to effectively handle repetitive tasks.

4.5.3 Functions

Create a function to calculate the average expenditure of customers. The function improves the reusability and organization of the code, making the analysis more organized and easier to maintain.

4.6 Additional analysis

A group analysis was conducted to calculate the average expenditure of different levels of education. The results further confirm that the consumption behavior of different customer groups is different.

The correlation analysis between income and total expenditure also supports the conclusion that the higher the income, the higher the expenditure.

4.7 Conclusion

EDA results provide valuable insights into customer behavior. Obviously, income and education level will affect the consumption pattern, and a small group of customers have made a significant contribution to the total income.

These insights can help companies formulate targeted marketing strategies, improve customer segmentation, and improve overall business performance.

Step 5: Hypothesis Testing and Machine Learning (ML) Analysis

5.1 Hypothesis Testing

In this step, we tested whether Income has a significant effect on customer response. Using an independent t-test, we compared the average income between customers who accepted (Response = 1) and those who did not (Response = 0).

Related code:

```

from scipy import stats

accepted = df[df['Response'] == 1]['Income']
not_accepted = df[df['Response'] == 0]['Income']

# t-test
t_stat, p_value = stats.ttest_ind(accepted, not_accepted, nan_policy='omit')

print("t-statistic:", t_stat)
print("p-value:", p_value)

t-statistic: 6.316420548976486
p-value: 3.2255685843410015e-10

```

Figure8: Code for Two-Sample t-Test on Income

Explanation:

- The t-test evaluates if the difference in mean income between the two groups is statistically significant.
- A small p-value (< 0.05) suggests that income is an important factor affecting customer response.

Business Insight:

Significant results indicate that marketing campaigns can be more effective if tailored to different income groups.

5.2 Machine Learning Analysis

To predict customer responses based on multiple features, we applied Logistic Regression, which is suitable for binary classification problems. The model uses predictors such as Income, Recency, MntWines, and MntMeatProducts.

Related code:

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# define the variable
X = df[['Income', 'Recency', 'MntWines', 'MntMeatProducts']]
y = df['Response']

# set train and test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# train model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

Accuracy: 0.8603603603603603

```

Figure9: Logistic Regression Model Code

Explanation:

1. Data was split into training and testing sets; the model was trained and evaluated for prediction accuracy.
2. Logistic Regression provides a probability-based prediction of whether a customer will respond positively.

Business Insight:

The model enables the business to target potential responders effectively, supporting data-driven marketing decisions.

5.3 Summary

Hypothesis testing highlighted the significance of individual variables, while Logistic Regression provided a predictive model for customer response. Together, these methods allow better targeting in marketing campaigns and improve overall business performance.